CISC 326 / CISC/CMPE 322 Software Architecture

March 13, 2026

Concrete Architecture

Group 25

Authors

Emanuel Caravaggio - 22gmx2@queensu.ca

Damon Clulow - 20djc9@queensu.ca

Raghab Dhungana - 21rd56@queensu.ca

Adam Elamin - 20aae5@queensu.ca

Paul Fernando - 20pf3@queensu.ca

Aditya Shetty - 21ass17@queensu.ca

## Abstract

In this report the concrete architecture of the Gemini CLI, an open source tool for interacting with Google's Gemini large language model from the command line, is analyzed. Using Scitool's Understand tool and inspecting the source code, the core components, their roles, and the dependencies between them were identified. This analysis revealed seven top-level components including the Terminal Interface, Orchestration Engine, Tool Execution Framework, External Integration Layer, Task Delegation System, Security and Execution Environment, and Configuration Subsystem. These were then broken down into subcomponents and analyzed from a high level perspective, to outline how they contribute to the operation of the Gemini CLI.Then, the architectural style of the concrete architecture is discussed and identified as a layered architecture that includes elements of a client-server interaction model using asynchronous and event-driven interactions between components. A reflexion analysis explores the discrepancies between the conceptual and concrete architecture revealing that many high level components were oversimplified leading to their breaking down into subcomponents in this report. Next, the Orchestration Engine component is explored in depth, decomposing it into submodules, and comparing its conceptual role with its concrete implementation. Finally, two use cases are defined and explored using a sequence diagram to outline the component interactions. Concrete method calls are included to provide a lower-level view at how different components work together so the Gemini CLI can perform tasks like project summarization, and automated issue triage.

## Table of Contents

**Table of Contents** ..... **2**
**Introduction and Overview** ..... **3**
**Derivation Process** ..... **3**
**Concrete Architecture** ..... **4**
- Components and Interactions ..... 4
- Architectural Styles ..... 7
- Concurrency ..... 8
- Reflexion Analysis ..... 8

**Subsystem Architecture** ..... **9**
**Use Cases** ..... **11**
- User Case 1 ..... 11
- Use Case 2 ..... 12

**Data Dictionary** ..... **13**
**Naming Conventions** ..... **13**
**Conclusions** ..... **14**
**Lessons Learnt** ..... **14**
**References** ..... **15**
**AI Collaboration Report** ..... **16**

# 1. Introduction and Overview

Gemini CLI is an open-source, cross-platform AI system that provides a command line interface for interacting with Google's Gemini models. Designed to be both a developer tool and an AI assistant, Gemini CLI allows users to submit prompts, retrieve information from their development environment, and execute actions such as web searches and shell commands through an integrated tool framework. While our conceptual architecture described the system at a high level, this report targets the concrete architecture of Gemini CLI, examining how the system is implemented within the Gemini CLI source code. To acquire the concrete architecture, we used the Understand tool to analyze the repository and create dependency graphs based on the project's directory structure. This examination helped determine that the system's organization is primarily contained within the packages/ directory. In this directory, the CLI package serves as the presentation layer and the Core package implements the primary orchestration logic. Additional subsystems such as tools, agent configuration models and external integrations support the system's functionality and extend its capabilities. This report presents the concrete architecture of Gemini CLI and describes the system's responsibilities and interactions of its major components. Additionally, it examines the architectural styles and concurrency model used by the system and displays a reflexion analysis, comparing the conceptual architecture with Gemini CLI's actual source code implementation. Lastly, the report analyzes the architecture of one of Gemini CLI's important subsystems, the Orchestration Engine, which demonstrates how user requests are processed and executed within the system

# 2. Derivation Process

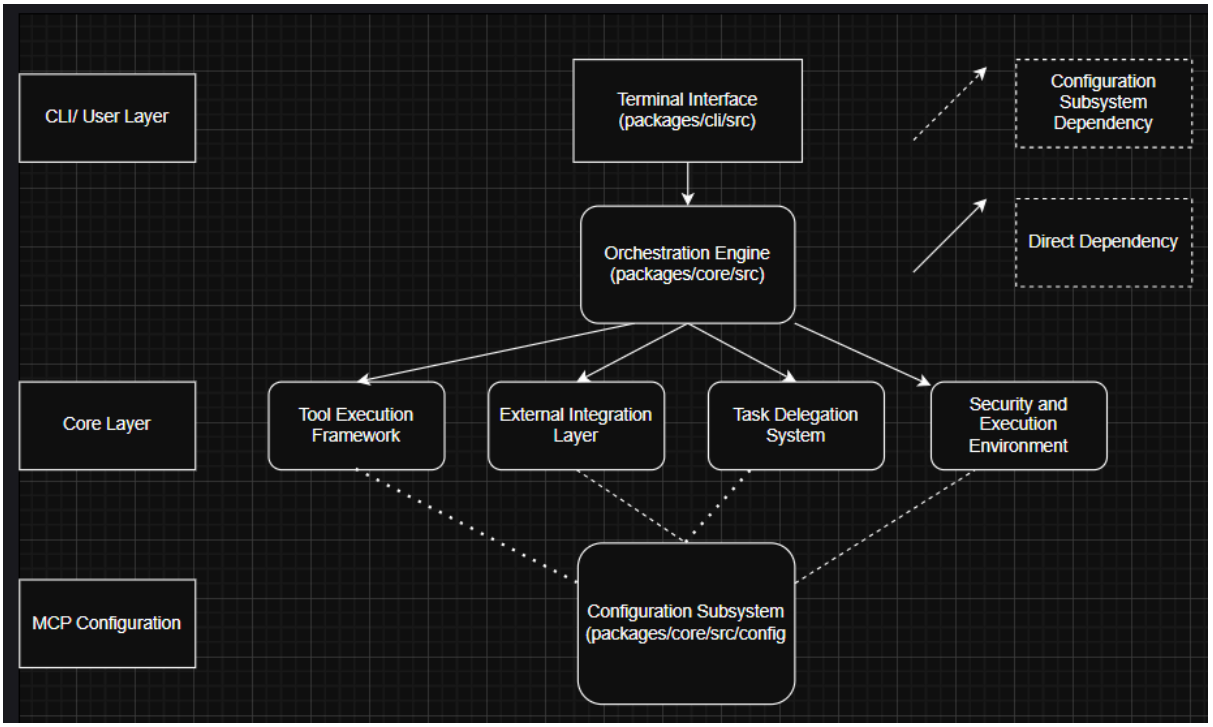
The derivation of the concrete architecture was an iterative process where we mapped our initial conceptual components directly to the physical file structure of the Gemini CLI repository. Using Scitools Understand, we generated dependency graphs to analyze the structure of Gemini. This bottom-up exploration revealed how several conceptual components were oversimplified in our conceptual architecture, such as the single Security block which was revealed through directory analysis to be a two-tiered system split between core/src/policy and core/src/safety. Through this process, we were able to verify the strict layered boundaries and refine the component hierarchy to reflect the actual code dependencies. To ensure the accuracy of this mapping, we cross-referenced the automated dependency graphs with some manual code inspections and official documentation. This process was critical in identifying hidden interactions that our conceptual model missed, such as the fact that the Tools Subsystem is architecturally nested within the Core Package rather than functioning as a peer component. This evidence-based approach allowed us to move from a theoretical interpretation of the system to a grounded, concrete architecture, ensuring that every interaction described in this report is supported by the physical reality of the Gemini CLI codebase.

# 3. Concrete Architecture

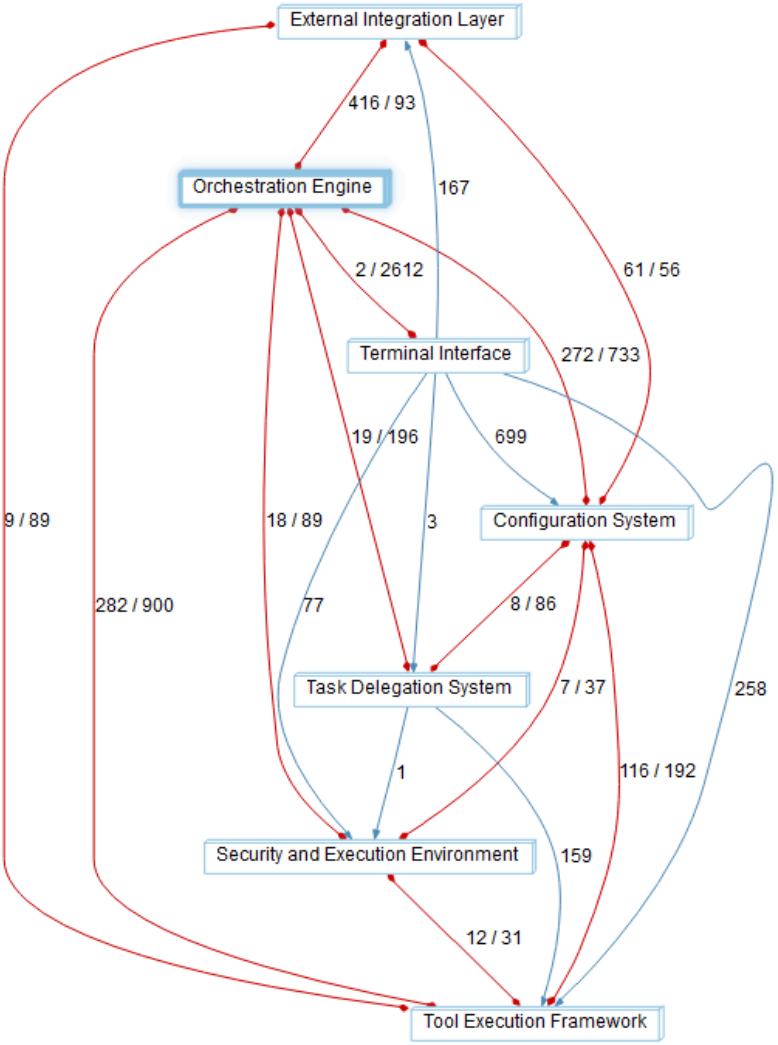
## 3.1 Components and Interactions

Using the Understand tool allowed us to look at the architecture of Gemini CLI, which in turn generated a dependency graph by directory structure. This process revealed that the system is organized within a single repository with all major subsystems contained within the packages/ directory. The Core Package (packages/core/) is the most important of top-level subsystems and is by far the most internally complex component. As discussed in A1, it serves as the broader logic layer, responsible for session management, prompt construction, state management, and orchestration of all interactions with the Gemini API. The src/ directory contains a set of dependent subcomponents that were not individually identified in the A1 conceptual architecture. These consist of utils and config for shared utilities and system configuration settings, a scheduler to manage sequence timing of operations, safety for abiding by content and safety policies, and agents to complete agentic reasoning loops as described in A1. This extends to commands that handle slash commands, Model Context Protocol integration and response formatting for outputs. Apart from components, the core also routes, directing requests to their appropriate sections, and checks for tool availability, while monitoring usage and performance. Finally, the core manages resources, events, skills for managing agent capabilities, and implementing safety mechanisms.

The CLI Package (packages/cli/) is the top-level presentation layer of the system. It accepts user input from the terminal and manages user-facing configuration. The CLI depends on the Core Package through its index.ts public interface to process prompts and manage the current state. It is a one-way dependency where the CLI depends on the Core, while the latter remains unaware. This property allows for cross-platform reliability and maintainability, consistent with the layered style identified in A1. The scripts package has 22 direct references to the CLI, which means that the CLI has many interdependent components tied to it specifically. The subsystems In Section 3.1 directly follow the structure of the following Box and Arrow Diagram.



The following dependency diagram shows the interactions between the top-level components, with system entry starting at the Terminal Interface which routes the user input to the Orchestration Engine. Then the Orchestration Engine directs the remaining top-level components to ensure that the correct response is performed in response to the user input. This explains where the Orchestration Engine has two-way dependencies with every other component.




## Terminal Interface

The Terminal Interface serves as the front end of the Gemini CLI. It accepts user input and routes it to the Orchestration Engine for processing and rendering responses for the user. This component is implemented in packages/cli and includes several modules to support user interaction.

- The rendering module, implemented in cli/src/ui contains the components and layouts which are required to ensure the interface is presented to the user clearly and consistently.
- The commands module, implemented in cli/src/commands contains the commands used by the UI to communicate with other components in the system. These include

commands for MCP server integration and installing or removing agent skills used to extend the functionality of the Gemini CLI.
- The configuration module, implemented in cli/src/config acts as a single source of truth for the UI settings.

## Orchestration Engine

The Orchestration Engine serves as the central component of the concrete architecture. It handles communication with the Gemini API, prompt creation, state management, and coordination of services to perform the required operations. It is implemented in core/src and is composed of several submodules.

- The prompts module, implemented in core/src/prompts, is responsible for formatting prompts for the Gemini API and for any additional services connected using MCP. This acts to format the user's request so that it will be processed correctly by external services.
- The routing module, implemented in core/src/routing, determines which model or execution path the request should be routed to.
- The services module, implemented in core/src/services contains various services that the orchestration logic may need to call on, including for managing state, getting context for the current interaction, and git related functionality.

## Tool Execution Framework

The Tool Execution Framework provides the Gemini model with tools for interacting with the environment. These tools gather information about the environment or perform other actions to help the model produce a more accurate response. Some examples of tools include file reading, web searches, or execution of shell commands. Depending on the tool the model plans to invoke, user permission may be required. There are two main subcomponents that make up the Tool Execution Framework.

- The tools subcomponent, implemented in core/src/tools acts as a tool repository. It contains the implementation of the available tools and executes them when invoked and returns the result.
- The scheduler subcomponent, implemented in core/src/scheduler, determines which tools are to be invoked and in what order, as multiple tools can not be invoked at once, essentially becoming an orchestrator for the Tool Execution Framework.

## External Integration Layer

The external integration layer component manages interactions with services and systems outside of the Gemini CLI architecture. The main focus of this component is to interact with extensions through MCP. Additionally, this component provides services to integrate the Gemini CLI with IDEs, including Visual Studio Code, to improve developer workflows. This component contains two main subcomponents.

- The MCP subcomponent, implemented in core/src/mcp, gives the Orchestration Engine access to external tool servers through MCP. It registers these external tools so that the Gemini model can invoke them as if they were tools from the Tool Execution

Framework. Included here is logic for OAuth, which ensures that appropriate authorization is acquired before accessing an external tool.

- The IDE subcomponent, implemented in core/src/ide, provides all the logic to detect the IDE and integrate with it. Including logic to find the IDE process, install the Gemini CLI extension, and get context from the IDE to add to user requests.

### Task Delegation System

The Task Delegation System component breaks down complex tasks by assigning agents to complete individual subtasks. This component is implemented in the core/src/agent directory, which includes the registry and implementation for agents that can complete specific types of tasks, including providing user support for interacting with the Gemini CLI. Additionally, this component includes the modules to invoke, manage and register agents that act within the Gemini CLI's environment.

### Security and Execution Environment

The Security and Execution Environment governs the tool and shell command execution to ensure the Gemini CLI does not take actions that may be unsafe or break constraints defined by the user. This component is broken down into two subcomponents.

- The policy subcomponent, implemented in core/src/policy, governs the execution of certain tools to determine if they require user confirmation before executing. This module has dependencies on all the top-level components, suggesting that it acts as an entry point to the Security and Execution Environment component.
- The safety subcomponent, implemented in core/src/safety, contains the logic needed to evaluate requests and to determine what must be done to ensure safe tool invocation. Its only dependency is on the Configuration Subsystem, suggesting that it uses user-set configuration settings and predetermined system rules to work with the policy subcomponent to determine what is required for every individual tool execution.

### Configuration Subsystem

The Configuration Subsystem is implemented in core/src/config, and manages the configuration for the logical behaviour of the Gemini CLI, governing how the tools, agents, and models are set up. Additionally, it helps persist information from previous conversations with the user. This component has many two-way dependencies, suggesting that it interacts with and is required for the correct operation of many of the top-level components, as well as facilitates communication between them.

## 3.2 Architectural Styles

The concrete architecture of Gemini CLI follows a primarily layered architecture style with elements of a client–server interaction model, which was close to the conceptual architecture. The system is organized within a single repository, with major subsystems located inside the packages/ directory. This layered organization separates the presentation layer (*packages/cli*), an Application/Orchestration Layer (*packages/core*), core orchestration logic, and supporting

subsystems that enable external integrations, tool execution, and system configuration. This enforces a tiered data processing flow where the CLI captures user input and relies on the Core package to manage the reasoning engine and session state. Analysis of the dependency graph reveals that while the CLI depends on the Core, the Core remains independent of the presentation logic, ensuring a one-way dependency that keeps the system independent of any specific platform. Although we observed non-adjacent dependencies where the CLI accesses shared configuration schemas in the Infrastructure Layer, the layered style still remains valid, as no lower-tier module depends on a higher-tier one.

## 3.3 Concurrency

In the Gemini CLI, the main concurrency model is asynchronous and event-driven. The Core module handles much of the logic, and the CLI module is responsible for interacting with the user. This allows for the user interface to remain responsive while high-latency operations are being performed. While the user interface remains active, the work is not executed in parallel, as multiple API requests or tool executions must be handled sequentially instead of being completed at the same time. This involves putting multiple operations on a queue, which, in the case of the Tool Execution Framework, is handled by the scheduler subcomponent.
While at a high level the Gemini CLI is asynchronous, some individual tools have internal parallelism to improve their performance. For example, file searching among multiple files is done concurrently, and additional utilities like ripgrep are implemented with multiple threads to ensure they are efficient. This asynchronous and event-driven design helps to decouple the presentation layer implemented in the CLI module from the orchestration logic implemented in Core. This ensures the UI can continue taking in user input and rendering output while it is waiting on the Core to complete its current task.

## 3.4 Reflexion Analysis

<u>High Level Reflexion Analysis:</u>
This explores the discrepancies between the conceptual and the concrete model of Gemini CLI.

**Divergence in Component Hierarchy**: In the conceptual model, the tools module was conceptualized as a primary top-level component, alongside the CLI and Core. However, looking at the concrete architecture, we are able to find that it was implemented as a subdirectory (packages/core/src/tools/) nested within the core package, indicating that the Core acts more as a centralized orchestrator rather than as a peer to the Tools system.

**One-Way Dependency**: In the conceptual model, we expected a layered structure where the CLI would depend on the Core for functionality while the Core remained independent of the CLI. The concrete architecture confirms this Layered Style as the CLI package has 22 direct references into the Core, but the Core remains unaware of the CLI. However, the dependency graph also shows that the internal structure of the Core is more complex than the simplified V-shaped data flow proposed in the conceptual model, with many internal interactions between Core components.

**The Orchestration vs. Agentic Shift**: The conceptual model focused on model reasoning within the Core. However, the concrete architecture reveals a dedicated Task Delegation system located in (core/src/agent). This indicates that the implementation relies on specialized agents to handle complex tasks, rather than relying solely on centralized reasoning within the Core.

2nd-Level Reflexion Analysis

A deeper analysis of the concrete architecture reveals how several high-level components were oversimplified in the conceptual architecture. While the conceptual model captured the system's major subsystems and overall layered structure, the use of Understand to analyze the dependency graphs and directory structure revealed additional internal modules and interactions that were not visible during the conceptual design phase.

**Tool Execution vs. Tool Scheduling**: In the conceptual architecture, we describe a general "suite of tools," the concrete architecture reveals a distinct Scheduler component (core/src/scheduler) responsible for coordinating tool execution.This subcomponent is essential for managing the asynchronous timing of operations, a functional requirement that was logically grouped but not architecturally defined in the conceptual report.

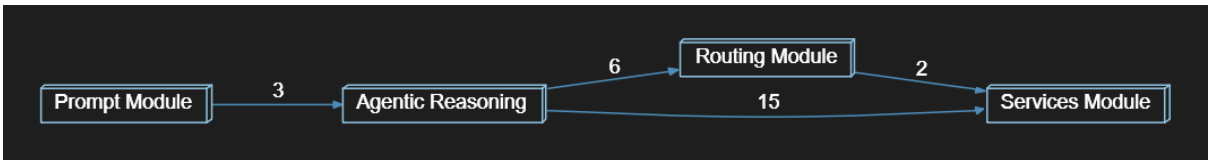
**Security Infrastructure**: The conceptual model referred to "safety and security checks" as a general responsibility within the system. However, the concrete architecture reveals a more structured implementation that separates this functionality into two distinct components: Policy and Safety. The Policy module (core/src/policy) governs tool execution permissions and determines whether certain actions require, acting as a gatekeeper that regulates access across multiple top-level components. The Safety module (core/src/safety) implements the logical rules and content filtering mechanisms used to evaluate requests and ensure they comply with system constraints.

# 4. Sub-System Architecture: Orchestration Engine

As defined previously, the Orchestration Engine serves as the central brain of the Gemini CLI. This subsystem is located within packages/core/src. It functions as a dispatcher, responsible for deconstructing high-level user prompts to actionable plans.

## 4.1 Sub-component Decomposition

There are four primary subcomponents that bridge the gap between user intent and tool execution: the Routing Module, the Services Module, the Prompts Module, and the Agentic Reasoning subcomponent. The following diagram, generated using the Understand tool, outlines the dependencies between the sub-components.



*Orchestration Engine Sub-components Dependencies*

### 4.1.1 Routing Module (src/routing/)

The Routing Module is the initial entry point for any request. It evaluates the user's prompt and other environmental context to determine the most efficient execution path. In terms of a "concrete" role, it implements selection logic that exists in classiferStrategy.ts and gemmaClassifierStrategy.ts to decide which model or sub-agent is appropriate for the task. It also maintains a one-way dependency with the Agentic Reasoning component, passing the routing decision as a prerequisite for the execution loop. However, it is not entirely isolated as it depends on services such as the modelConfigService.ts (Services Module) to understand system capabilities when making routing decisions. [5]

### 4.1.2 Prompts Module (src/prompts/)

This module serves as a template repository that provides context and formatting for model interactions. It uses promptProvider.ts and prompt-registry.ts to manage system instructions and model-specific snippets (e.g., mcp-prompts.ts). It is essentially a resource that is consumed primarily by the Agentic Reasoning component to synthesize final model payloads. Specifically, the prompt synthesis in promptProvider.ts is dynamic, adjusting instructions based on specific AgentType or tool-use state currently active in the executor. [5]

### 4.1.3 Agentic Reasoning (src/agents/src/scheduler/)

This is essentially the execution core of the engine, it manages the lifecycle of the execution loop and delegates specialized sub-tasks to agents. This sub-component is implemented through agent-scheduler.ts and local-executor.ts. It instantiates specialised agents such as codebase-investigator.ts or generalist-agent.ts depending on the instruction received from the router. This sub-component receives input from the routing component but is also a consumer of the Services Modules. To maintain modularity, the agent-scheduler.ts calls into the modelRouterService.ts (Routing Module) to determine if a task requires a high-reasoning model or a faster, cheaper alternative. [5]

### 4.1.4 Services Module (src/services/)

This module provides the environmental context and utilities for the engine to operate. It includes critical services like contextManager.ts (history tracking), fileDiscoveryService.ts (codebase awareness), and executionLifecycleService.ts (event monitoring). It serves as a provider for the Routing Module, and provides bidirectional traffic with the Agentic Reasoning component, acting as short-term memory that agents will update and query during a task. [5]

## 4.2 Orchestration Engine: Conceptual vs. Concrete Architecture

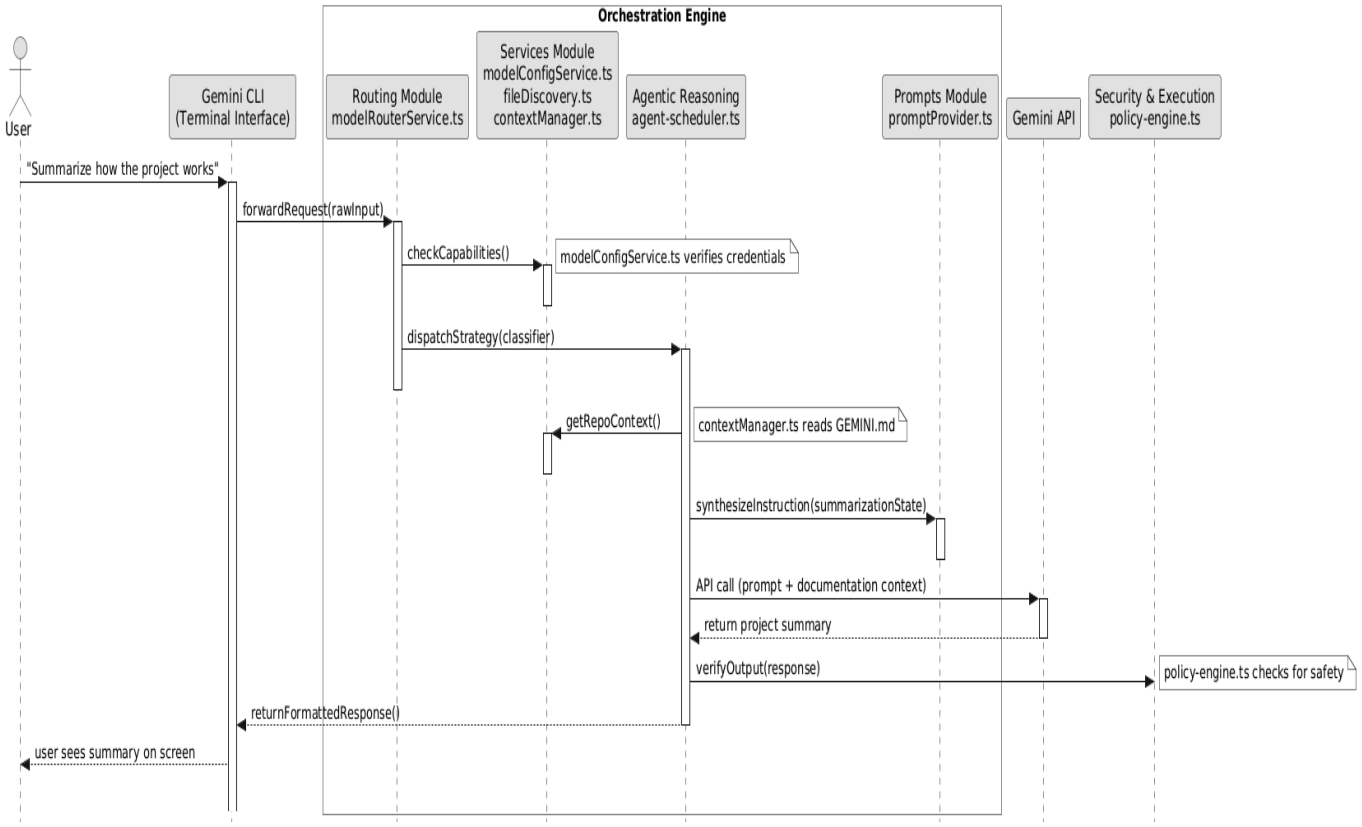
In the conceptual architecture, the Orchestration engine was represented as a single, monolithic component within the Application Layer. It was tasked with general "backend orchestration" and "API communication". The actual implementation is far more granular, comprising the aforementioned four sub-components. A significant divergence is the presence of the Routing Module (src/routing/). The conceptual model did not account for the

complex intent classification and strategy selection logic required to handle multi-model environments.[4]

The conceptual module proposed a stricter Layered Architecture style, where the Orchestration Engine (Application Layer) depends on the Service and Infrastructure Layers in a top-down fashion. However, the Prompts Module, which would conceptually be a passive resource in the configuration or service layer, has three concrete dependencies on the Agentic Reasoning component. This indicates that the instruction synthesis is not a static pull operation but is driven by the state of the active agent as well, contradicting the top-down flow envisioned in the conceptual architecture. [4]

# 5. Use Cases

## 5.1 Use Case: Project Summarization Sequence Diagram

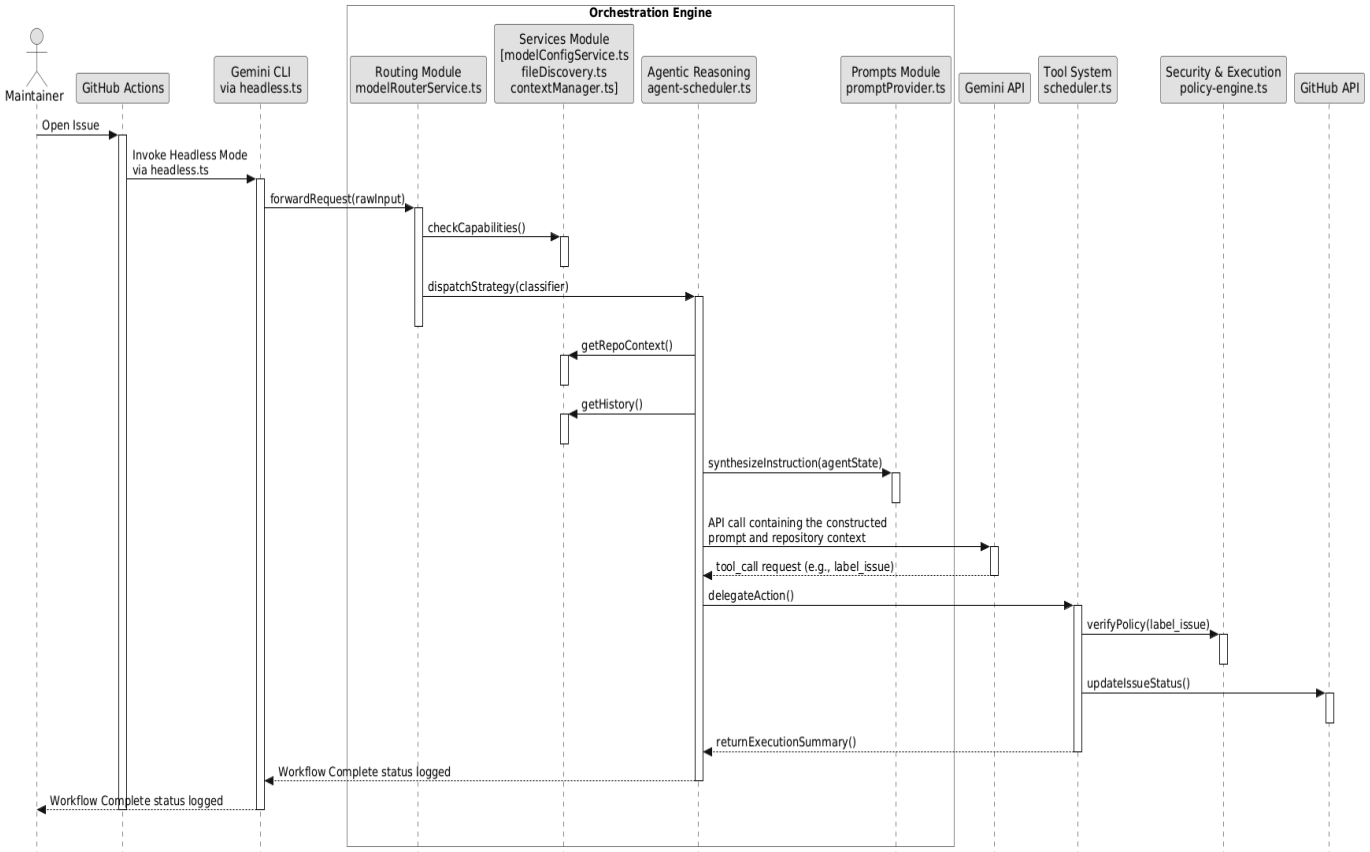



Interaction Summary

- The User enters a prompt into the terminal to start the process.
- The Gemini CLI receives the entry and forwards the user input to the Orchestration Engine.
- The Agentic Reasoning module retrieves project context (GEMINI.md) from the Services Module.
- The Orchestration Engine acts as the orchestrator, sending the prompt to the Gemini API for processing.
- The Gemini API returns a textual response back to the Orchestration Engine.

- The Orchestration Engine passes the final response back to the Gemini CLI.

## 5.2 Use Case: Automated Issue Triage and Code Suggestion




Interaction Summary

- Process begins when the maintainer opens an issue, triggering GitHub Actions to invoke the Gemini CLI in Headless Mode using headless.ts.
- The CLI forwards the raw input to the Routing Module (modelRouterService.ts). It verifies available models and credentials with the Services Module (modelConfigService.ts), before dispatching the classification strategy to the Agentic Reasoning module.
- The Agentic Reasoning module pulls repository context and conversation history from the Services Module. The Prompts Module pulls the current agent state to generate a dynamic system prompt.
- The Agentic Reasoning module sends the constructed prompt and repository context to the Gemini API, which then analyzes the input and returns a specific tool call request (e.g., label_issue).
- The Agentic Reasoning module delegates the requested action to the Tool System. The Tool system verifies the action against the Security & Execution module to ensure it is safe and approved. Once cleared, it updates the issue status via the GitHub API.

- After successful execution, the Agentic Reasoning module returns an execution summary to the Gemini CLI. The CLI logs a “workflow complete” status back to the GitHub Actions environment.

# 6. Data Dictionary

| Component | Type | Description |
|---|---|---|
| packages/cli/ | Directory | Presentation Layer. Handles IO presentation and user configuration. |
| packages/core/ | Directory | Backend Logic Layer. Manages most of the complex prompt disassembly and state management. |
| packages/core/src/tools | Directory | Manages the Tool Subsystem. Contains all memory for shell tools and MCP implementation. |
| packages/vscode-ide-companion | Directory | Extends VSCode Accessibility to the CLI. |
| packages/a2a-server/ | Directory | Implements Agentic communication. |
| agents/ | SubDirectory | Implements Agentic Reasoning and task planning for the core. |
| mcp/ | SubDirectory | Manages Model Context Protocol connections to outside systems. |
| services/ | SubDirectory | Condenses all incoming communication and responses. |
| safety | SubDirectory | Enforces Google safety policies before tool invocation. |
| index.ts | Typescript File | API entry point of the core. Interface between the CLI and the Core. |

# 7. Naming Conventions

- **Ripgrep:** A search tool that looks for and returns lines in files in the current directory that contain a provided regex pattern.
- **Visual Studio Code:** An IDE developed by Microsoft, supports many extensions to add additional functionality to the IDE.
- **IDE:** Integrated development environment, software that provides a variety of features, to write, run, edit, test, and debug code.

- **OAuth:** An authorization framework that gives a program the ability to interact with other programs on behalf of the user.

# 8. Conclusions

In conclusion, the concrete architecture of Gemini CLI largely supports the layered structure we proposed in our conceptual architecture while revealing additional implementation details and dependencies within the system. Through our analysis using the Understand tool, we showed that the CLI package functions as the presentation layer, while the Core package acts as a central orchestration component, responsible for managing prompt construction, system state, and interaction with the Gemini API. Supporting subsystems such as tools, agent configuration models, and external integrations extend the system's functionality and enable it to interact with both local and external environments. The reflexion analysis exposed divergences from our conceptual model, including the integration of the tools subsystem within the Core, the presence of a dedicated Task Delegation system, and the more complex internal interactions between core components. Through examining the Orchestration Engine subsystem, we revealed its important subcomponents, such as the agentic reasoning and the routing, prompts and services modules. These components work together to perform important tasks within the system, like processing user prompts, managing system states, and executing tasks. Overall, the concrete architecture demonstrates that Gemini CLI is built on a modular and extensible design that supports complex interactions between AI models, tools and external services. While our conceptual architecture provided an effective understanding of the high-level view of Gemini CLI, the concrete architecture offers a clear understanding of the system’s operations in practice.

# 9. Lessons Learnt

Through the completion of this report, the team gained practical experience in mapping a conceptual architecture to its concrete implementation. The Understand tool allowed the team to look deeper at the Gemini CLI architecture and generate subsequent dependency graphs. This process revealed discrepancies and oversimplifications made by conceptual models. Specifically, we discovered that the Tools Subsystem, previously thought to be an independent peer, is actually nested within the Core package. It was also revealed that the security infrastructure is not a single block but a two-tiered system divided into policy and safety modules. Additionally, the identification of a dedicated scheduler emphasized the complexity of managing asynchronous tool execution. Overall, this second deliverable emphasized the need to analyze physical dependencies to truly understand system behaviour, beyond conceptual predictions. The concrete architecture exposed the intricate intent classification logic within the Routing Module and the dense two-way dependencies of the Configuration Subsystem. Completion of this assignment reinforced the importance of continuous architectural reflection when confronting the realities of strict policy enforcement, state management, and execution environments.

# References

[1] PlantUML. “PlantUML Software.” plantuml.com. [Online]. Available: https://editor.plantuml.com/. (Accessed: Mar. 11, 2024).

[2] Gemini CLI Documentation. [Online]. Available: https://geminicli.com/docs/. [Accessed: Mar. 3, 2026].

[3] Gemini CLI Developers, "Automation and triage processes," Gemini CLI Documentation, 2026. [Online]. Available: https://geminicli.com/docs/issue-and-pr-automation/. [Accessed: Mar. 6, 2026].

[4] Gemini CLI - Conceptual Architecture Summary. [Online]. Available: https://onq.queensu.ca/d2l/le/content/1132545/viewContent/6711621/View. [Accessed: Mar. 6, 2026].

[5] gemini-cli. [Online Repository]. Available: https://github.com/google-gemini/gemini-cli. [Accessed: Mar. 6, 2026].

[6] A. Gallant, BurntSushi/ripgrep. Rust. [Online]. Available: https://github.com/BurntSushi/ripgrep. [Accessed: Mar. 10, 2026].

[7] “What Is OAuth? | Microsoft Security.” [Online]. Available: https://www.microsoft.com/en-us/security/business/security-101/what-is-oauth. [Accessed: Mar. 10, 2026].

# AI Collaboration Report

Regarding our use of AI to aid us in our understanding of the concrete architecture of Gemini CLI, we continued to use OpenAI's LLM ChatGPT - 5.2, February 2026 version, as well as Google's Gemini 3, November 2025 version, just as we did to help understand the conceptual architecture. We opted to continue to use these LLMs for their ability to interpret documentation, explain system functionality and identify relationships between implementation components. We chose to continue our use of these LLMs for their ability to aid in the identification of relationships between components in the system implementation. When given sufficient context, the LLMs can help interpret how different modules interact, how control flows through the system during execution, and how external services are accessed. This allows us to better understand how the conceptual components we outlined previously are realized in the actual implementation of the system through items such as files, packages and command handlers. While the capabilities of the LLMs we chose may not be specific to them, the LLMs are widely used, easily accessible, and appropriate for the tasks we wanted our AI teammate to perform in the context of the deliverable.
To ensure responsible use of AI tools, our group used a clear methodology for interacting with the models. All group members were allowed to collaboratively develop prompts, but we restricted the tasks assigned to the LLMs so that they served as support tools rather than replacing the analytical work of the team. The LLMs primary purposes included:

1. Identifying potential implementation responsibilities of modules or files regarding certain features like command parsing or API communication. We believed this was an appropriate use of the LLMs, as it was given extensive information about the system and code from reliable sources before being prompted, and it was not generating original analysis for us, rather just interpreting already given documentation provided by us or directly by Gemini CLI itself.
2. Helping verify assumptions made by group members regarding the concrete architecture of the system, granted that the group was consulted about the assumption already. This served as an additional perspective when interpreting the system architecture, which we deemed appropriate since all group members were consulted before the use and necessary information about the system was provided before prompting it to verify an assumption.

Our prompting strategy remained quite similar to what we did in our understanding of the conceptual architect as we focused on providing sufficient context and restricting the models responses to the information we supplied. To do this we included direct text from the documentation and relevant code snippets directly into prompts. An example of a critical prompt we used was:

- We are analyzing the concrete architecture of the Gemini CLI system for a software architecture course. Based only on the source code and documentation provided

above, identify which modules or files appear responsible for communication with the Gemini API.

To further refine this prompt we added further constraints through specifying that we are undergraduate students analyzing the system and instructing the LLM to only use the information provided in the prompt. If the output was deemed by the group as too general or unclear, we disregarded the output or tried to reiterate on the prompt asking for a more detailed explanation.

To ensure the output of our AI team member was void of factual errors, biases or hallucinations we implemented precautions to verify its contributions. First we ensured that all documentation given to the LLMs was directly from reliable sources, including the official Gemini CLI documentation and system source code. This helped us ensure that the AIs responses were grounded in accurate information. To further verify output, all responses by the LLMs were carefully reviewed by group members instead of being immediately used to inform of understanding of the system. Along with this any AI explanations that could have possibly influenced our understanding of the system was also cross-referenced with course materials, which ensured that our architectural interpretations remained consistent with the concepts discussed.

Based on the recorded uses of our AI teammate we believe that the LLMs we used contributed an equal amount of work to this deliverable as it did to our previous work, meaning that it contributed approximately 10% of the final deliverable. While the LLMs were useful for identifying responsibilities of components and verifying assumptions, the majority of the work was performed by the members of our group and through the use of the Understand tool. Most of our effort during the deliverable involved examining the system, discussing interpretations of the systems architecture and preparing final diagrams and explanations for the deliverable. Since our AI teammates' role was limited to assisting with interpretation and validation, it mainly acted as a supporting role in the work helping to clarify documentation when necessary and provide verification when provided an assumption about the systems architecture. Due to its assistant role we believe its contribution was smaller than that of any individual group member.

The use of our AI teammate in our group's workflow had both advantages and challenges. One benefit was the LLMs ability to provide clarification regarding documentation and system behaviour when necessary. This allowed our group to develop a shared understanding of the system more efficiently when interpreting certain implementation details. Working with our AI teammate also introduced negative effects regarding our workflow. Since our methodology required careful prompt construction and significant verification of the outputs, preparing and validating responses sometimes required more effort and time compared to doing our own analysis. Additionally, it also played an effective role in settling group differences, leading us to stray away from disagreement, which was important under the time constraints of the deliverable. Through our second use of our AI teammate, we believe we further refined our use of AI in our understanding of Gemini CLI. Through our AI teammates’ aid we learned how through clear prompting and validation of outputs, AI can be a great assistant for analysis to help better our own explanation of the concrete architecture of Gemini CLI.